



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΥΠΟΛΟΓΙΣΤΙΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΛΕΙΤΟΥΡΓΙΚΑ ΣΥΣΤΗΜΑΤΑ ΥΠΟΛΟΓΙΣΤΩΝ
3^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΜΗΧΑΝΙΣΜΟΙ ΕΙΚΟΝΙΚΗΣ ΜΝΗΜΗΣ

Άσκηση 1.1 - Κλήσεις Συστήματος και Βασικοί Μηχανισμοί του Λειτουργικού Συστήματος για τη Διαχείριση της Εικονικής Μνήμης

Στο παρόν μέρος της 3^{ης} εργαστηριακής άσκησης, μέσω της κλήσης συστήματος `mmap()`, πραγματοποιήσαμε δέσμευση `private` και `shared` σελίδων μνήμης, καθώς και απεικόνιση ενός αρχείου στον εικονικό χώρο διευθύνσεων. Παρακολουθώντας την αντιστοίχιση εικονικών και φυσικών διευθύνσεων, επιβεβαιώσαμε τη λειτουργία της Σελιδοποίησης κατ' απαίτηση (Demand Paging), καθώς η φυσική μνήμη (RAM) εκχωρούνταν μόνο κατά την πρώτη προσπάθεια εγγραφής. Στη συνέχεια, με τη χρήση της κλήσης `fork()`, αναλύσαμε τη διαχείριση της μνήμης μεταξύ γονικής και θυγατρικής διεργασίας. Εκεί παρατηρήσαμε τον μηχανισμό Copy - on - Write (CoW) στην ιδιωτική μνήμη, όπου το λειτουργικό σύστημα δημιουργεί ένα νέο φυσικό αντίγραφο της σελίδας μόνο όταν υπάρξει τροποποίηση, σε αντίθεση με τη διαμοιραζόμενη μνήμη όπου και οι δύο διεργασίες έδειχναν σταθερά στο ίδιο φυσικό πλαίσιο. Τέλος, εφαρμόσαμε έλεγχο δικαιωμάτων μνήμης με την `mprotect()` και αποδεσμεύσαμε τους πόρους μέσω της `munmap()`.

Ο τροποποιημένος από εμάς κώδικας του αρχείου `mmap.c`, παρουσιάζεται παρακάτω.

```
angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 134x60
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <sys/mman.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <errno.h>
#include <stdint.h>
#include <signal.h>
#include <sys/wait.h>

#include "help.h"

#define RED    "\033[31m"
#define RESET  "\033[0m"

char *heap_private_buf;
char *heap_shared_buf;

char *file_shared_buf;

uint64_t buffer_size;

/*
 * Child process' entry point.
 */
void child(void) {
    uint64_t pa;

//DONE

    /*
     * Step 7 - Child
     */
    if (0 != raise(SIGSTOP))
        die("raise(SIGSTOP)");
    printf("\n[CHILD] Virtual Memory Map:\n");
    show_maps();

//DONE

    /*
     * Step 8 - Child
     */
    if (0 != raise(SIGSTOP))
        die("raise(SIGSTOP)");
    pa = get_physical_address((uintptr_t)heap_private_buf);
    printf("[CHILD] Physical address of heap_private_buf: 0x%lx\n", pa);

//DONE

    /*
     * Step 9 - Child
```

angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 134x60

```
*/
if (0 != raise(SIGSTOP))
    die("raise(SIGSTOP)");
heap_private_buf[0] = 'C';
pa = get_physical_address((uintptr_t)heap_private_buf);
printf("[CHILD] Physical address of heap_private_buf AFTER write: 0x%x\n", pa);

//DONE

/*
 * Step 10 - Child
 */

if (0 != raise(SIGSTOP))
    die("raise(SIGSTOP)");
heap_shared_buf[0] = 'S';
pa = get_physical_address((uintptr_t)heap_shared_buf);
printf("[CHILD] Physical address of heap_shared_buf AFTER write: 0x%x\n", pa);

//DONE

/*
 * Step 11 - Child
 */

if (0 != raise(SIGSTOP))
    die("raise(SIGSTOP)");
if (mprotect(heap_shared_buf, buffer_size, PROT_READ) == -1) die("mprotect");
printf("[CHILD] Verifying maps (Child should now have r-- permissions on shared buffer):\n");
show_maps();

//DONE

/*
 * Step 12 - Child
 */

munmap(heap_private_buf, buffer_size);
munmap(heap_shared_buf, buffer_size);
}

/*
 * Parent process' entry point.
 */

void parent(pid_t child_pid) {
    uint64_t pa;
    int status;
    /* Wait for the child to raise its first SIGSTOP. */
    if (-1 == waitpid(child_pid, &status, WUNTRACED))
        die("waitpid");

    //DONE

    /*
     * Step 7: Print parent's and child's maps. What do you see?
     * Step 7 - Parent
     */
}
```

angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 134x60

```
printf(RED "\nStep 7: Print parent's and child's map.\n" RESET);
press_enter();
printf("[PARENT] Virtual Memory Map:\n");
show_maps();
if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, WUNTRACED))
    die("waitpid");

//DONE

/*
 * Step 8: Get the physical memory address for heap_private_buf.
 * Step 8 - Parent
 */

printf(RED "\nStep 8: Find the physical address of the private heap "
        "buffer (main) for both the parent and the child.\n" RESET);
press_enter();
pa = get_physical_address((uintptr_t)heap_private_buf);
printf("[PARENT] Physical address of heap_private_buf: 0x%x\n", pa);

if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, WUNTRACED))
    die("waitpid");

//DONE

/*
 * Step 9: Write to heap_private_buf. What happened?
 * Step 9 - Parent
 */

printf(RED "\nStep 9: Write to the private buffer from the child and "
        "repeat step 8. What happened?\n" RESET);
press_enter();
pa = get_physical_address((uintptr_t)heap_private_buf);
printf("[PARENT] Physical address of heap_private_buf: 0x%x\n", pa);
if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, WUNTRACED))
    die("waitpid");

//DONE

/*
 * Step 10: Get the physical memory address for heap_shared_buf.
 * Step 10 - Parent
 */

printf(RED "\nStep 10: Write to the shared heap buffer (main) from "
        "child and get the physical address for both the parent and "
        "the child. What happened?\n" RESET);
press_enter();
pa = get_physical_address((uintptr_t)heap_shared_buf);
printf("[PARENT] Physical address of heap_shared_buf: 0x%x\n", pa);

if (-1 == kill(child_pid, SIGCONT))
```

angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 134x60

```
die("kill");
if (-1 == waitpid(child_pid, &status, WUNTRACED))
    die("waitpid");

//DONE

/*
 * Step 11: Disable writing on the shared buffer for the child
 * (hint: mprotect(2)).
 * Step 11 - Parent
 */

printf(RED "\nStep 11: Disable writing on the shared buffer for the "
        "child. Verify through the maps for the parent and the "
        "child.\n" RESET);
press_enter();
printf("\n[PARENT] Verifying maps (Parent should still have rw- permissions):\n");
show_maps();
if (-1 == kill(child_pid, SIGCONT))
    die("kill");
if (-1 == waitpid(child_pid, &status, 0))
    die("waitpid");

//DONE

/*
 * Step 12: Free all buffers for parent and child.
 * Step 12 - Parent
 */
munmap(heap_private_buf, buffer_size);
munmap(heap_shared_buf, buffer_size);
}

int main(void) {
    pid_t mypid, p;
    int fd = -1;
    uint64_t pa;
    mypid = getpid();
    buffer_size = 1 * get_page_size();

//DONE

/*
 * Step 1: Print the virtual address space layout of this process.
 */

printf(RED "\nStep 1: Print the virtual address space map of this "
        "process [%d].\n" RESET, mypid);
press_enter();
show_maps();

//DONE

/*
 * Step 2: Use mmap to allocate a buffer of 1 page and print the map
 * again. Store buffer in heap_private_buf.
 */

printf(RED "\nStep 2: Use mmap(2) to allocate a private buffer of "
        "size equal to 1 page and print the VM map again.\n" RESET);
```

angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 134x60

```
press_enter();
heap_private_buf = mmap(NULL, buffer_size, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
if (heap_private_buf == MAP_FAILED) {
    die("mmap private buffer failed");
}
show_maps();

//DONE

/*
 * Step 3: Find the physical address of the first page of your buffer
 * in main memory. What do you see?
 */

printf(RED "\nStep 3: Find and print the physical address of the "
        "buffer in main memory. What do you see?\n" RESET);
press_enter();
pa = get_physical_address((uintptr_t)heap_private_buf);
printf("Physical address of heap_private_buf: 0x%lx\n", pa);

//DONE

/*
 * Step 4: Write zeros to the buffer and repeat Step 3.
 */

printf(RED "\nStep 4: Initialize your buffer with zeros and repeat "
        "Step 3. What happened?\n" RESET);
press_enter();
memset(heap_private_buf, 0, buffer_size);
pa = get_physical_address((uintptr_t)heap_private_buf);
printf("Physical address of heap_private_buf after memset: 0x%lx\n", pa);

//DONE

/*
 * Step 5: Use mmap(2) to map file.txt (memory-mapped files) and print
 * its content. Use file_shared_buf.
 */

printf(RED "\nStep 5: Use mmap(2) to read and print file.txt. Print "
        "the new mapping information that has been created.\n" RESET);
press_enter();
fd = open("file.txt", O_RDONLY);
if (fd == -1)
    die("open file.txt");
struct stat st;
if (fstat(fd, &st) == -1)
    die("fstat");
file_shared_buf = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
if (file_shared_buf == MAP_FAILED)
    die("mmap file");
printf("--- File Content ---\n");
write(STDOUT_FILENO, file_shared_buf, st.st_size);
printf("\n-----\n");
show_maps();

//DONE
```



```

/*
 * Step 6: Use mmap(2) to allocate a shared buffer of 1 page. Use
 * heap_shared_buf.
 */

printf(RED "\nStep 6: Use mmap(2) to allocate a shared buffer of size "
        "equal to 1 page. Initialize the buffer and print the new "
        "mapping information that has been created.\n" RESET);

press_enter();
heap_shared_buf = mmap(NULL, buffer_size, PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);
if (heap_shared_buf == MAP_FAILED)
    die("mmap shared");
memset(heap_shared_buf, 0, buffer_size);
show_maps();
p = fork();
if (p < 0)
    die("fork");
if (p == 0) {
    child();
    return 0;
}
parent(p);
if (-1 == close(fd))
    perror("close");
return 0;
}

```

Εκτελώντας τον κώδικα, πήραμε το ακόλουθο output.

```

angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 134x60
oslab067@os-node1:~/lab3_exercise/PartA/mmap$ ls
Makefile file.txt help.c help.h mmap mmap.c mmap.c.backup
oslab067@os-node1:~/lab3_exercise/PartA/mmap$ ./mmap

Step 1: Print the virtual address space map of this process [1625527].

Virtual Memory Map of process [1625527]:
55be8a838000-55be8a839000 r--p 00000000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a839000-55be8a83a000 r-xp 00001000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83a000-55be8a83b000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83b000-55be8a83c000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83c000-55be8a83d000 rw-p 00003000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8b80d000-55be8b82e000 rw-p 00000000 00:00 0 [heap]
7f2a36438000-7f2a3645a000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a3645a000-7f2a365b3000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a365b3000-7f2a36602000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36602000-7f2a36606000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36606000-7f2a36608000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36608000-7f2a3660e000 rw-p 00000000 00:00 0
7f2a36614000-7f2a36615000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36615000-7f2a36635000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36635000-7f2a3663d000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663e000-7f2a3663f000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663f000-7f2a36640000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36640000-7f2a36641000 rw-p 00000000 00:00 0
7ffe92d0c000-7ffe92d2d000 rw-p 00000000 00:00 0 [stack]
7ffe92db3000-7ffe92db7000 r--p 00000000 00:00 0 [vvar]
7ffe92db7000-7ffe92db9000 r-xp 00000000 00:00 0 [vdso]
-----

Step 2: Use mmap(2) to allocate a private buffer of size equal to 1 page and print the VM map again.

Virtual Memory Map of process [1625527]:
55be8a838000-55be8a839000 r--p 00000000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a839000-55be8a83a000 r-xp 00001000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83a000-55be8a83b000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83b000-55be8a83c000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83c000-55be8a83d000 rw-p 00003000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8b80d000-55be8b82e000 rw-p 00000000 00:00 0 [heap]
7f2a36438000-7f2a3645a000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a3645a000-7f2a365b3000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a365b3000-7f2a36602000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36602000-7f2a36606000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36606000-7f2a36608000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36608000-7f2a3660e000 rw-p 00000000 00:00 0
7f2a36614000-7f2a36615000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36615000-7f2a36635000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36635000-7f2a3663d000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663d000-7f2a3663e000 rw-p 00000000 00:00 0
7f2a3663e000-7f2a3663f000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663f000-7f2a36640000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36640000-7f2a36641000 rw-p 00000000 00:00 0
7ffe92d0c000-7ffe92d2d000 rw-p 00000000 00:00 0 [stack]
7ffe92db3000-7ffe92db7000 r--p 00000000 00:00 0 [vvar]
7ffe92db7000-7ffe92db9000 r-xp 00000000 00:00 0 [vdso]
-----

```

```
angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cs.lab.ece.ntua.gr — 134x60
```

Step 3: Find and print the physical address of the buffer in main memory. What do you see?

```
VA[0x7f2a3663d000] is not mapped; no physical memory allocated.  
Physical address of heap_private_buf: 0x0
```

Step 4: Initialize your buffer with zeros and repeat Step 3. What happened?

```
Physical address of heap_private_buf after memset: 0x1d96df000
```

```
angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cs.lab.ece.ntua.gr — 134x60
```

Step 5: Use mmap(2) to read and print file.txt. Print the new mapping information that has been created.

```
--- File Content ---  
Hello everyone!
```

```
-----  
Virtual Memory Map of process [1625527]:  
55be8a838000-55be8a839000 r--p 00000000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8a839000-55be8a83a000 r-xp 00001000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8a83a000-55be8a83b000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8a83b000-55be8a83c000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8a83c000-55be8a83d000 r--p 00003000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8b80d000-55be8b82e000 rw-p 00000000 00:00 0 [heap]  
7f2a36438000-7f2a3645a000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a3645a000-7f2a365b3000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a365b3000-7f2a36602000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a36602000-7f2a36606000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a36606000-7f2a36608000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a36608000-7f2a3660e000 rw-p 00000000 00:00 0  
7f2a36613000-7f2a36614000 r--p 00000000 00:27 1444948 /home/oslab/oslab067/lab3_exercise/PartA/mmap/file.txt  
7f2a36614000-7f2a36615000 r-xp 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a36615000-7f2a36635000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a36635000-7f2a3663d000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a3663d000-7f2a3663e000 rw-p 00000000 00:00 0  
7f2a3663e000-7f2a3663f000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a3663f000-7f2a36640000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a36640000-7f2a36641000 rw-p 00000000 00:00 0  
7ffe92d0c000-7ffe92d2d000 rw-p 00000000 00:00 0 [stack]  
7ffe92db3000-7ffe92db7000 r--p 00000000 00:00 0 [vvar]  
7ffe92db7000-7ffe92db9000 r-xp 00000000 00:00 0 [vdso]  
-----
```

```
angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cs.lab.ece.ntua.gr — 134x60
```

Step 6: Use mmap(2) to allocate a shared buffer of size equal to 1 page. Initialize the buffer and print the new mapping information that has been created.

```
Virtual Memory Map of process [1625527]:  
55be8a838000-55be8a839000 r--p 00000000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8a839000-55be8a83a000 r-xp 00001000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8a83a000-55be8a83b000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8a83b000-55be8a83c000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8a83c000-55be8a83d000 rw-p 00003000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap  
55be8b80d000-55be8b82e000 rw-p 00000000 00:00 0 [heap]  
7f2a36438000-7f2a3645a000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a3645a000-7f2a365b3000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a365b3000-7f2a36602000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a36602000-7f2a36606000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a36606000-7f2a36608000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so  
7f2a36608000-7f2a3660e000 rw-p 00000000 00:00 0  
7f2a36612000-7f2a36613000 rw-s 00000000 00:01 10429 /dev/zero (deleted)  
7f2a36613000-7f2a36614000 r--p 00000000 00:27 1444948 /home/oslab/oslab067/lab3_exercise/PartA/mmap/file.txt  
7f2a36614000-7f2a36615000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a36615000-7f2a36635000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a36635000-7f2a3663d000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a3663d000-7f2a3663e000 rw-p 00000000 00:00 0  
7f2a3663e000-7f2a3663f000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a3663f000-7f2a36640000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so  
7f2a36640000-7f2a36641000 rw-p 00000000 00:00 0  
7ffe92d0c000-7ffe92d2d000 rw-p 00000000 00:00 0 [stack]  
7ffe92db3000-7ffe92db7000 r--p 00000000 00:00 0 [vvar]  
7ffe92db7000-7ffe92db9000 r-xp 00000000 00:00 0 [vdso]  
-----
```


angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cs.lab.ece.ntua.gr — 134x60

Step 7: Print parent's and child's map.

[PARENT] Virtual Memory Map:

Virtual Memory Map of process [1625527]:

```
55be8a838000-55be8a839000 r--p 00000000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a839000-55be8a83a000 r-xp 00001000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83a000-55be8a83b000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83b000-55be8a83c000 r--p 00003000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83c000-55be8a83d000 rw-p 00004000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8b80d000-55be8b82e000 rw-p 00000000 00:00 0 [heap]
7f2a36438000-7f2a3645a000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a3645a000-7f2a365b3000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a365b3000-7f2a36602000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36602000-7f2a36606000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36606000-7f2a36608000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36608000-7f2a3660e000 rw-p 00000000 00:00 0
7f2a36612000-7f2a36613000 rw-s 00000000 00:01 10429 /dev/zero (deleted)
7f2a36613000-7f2a36614000 r--p 00000000 00:27 1444948 /home/oslab/oslab067/lab3_exercise/PartA/mmap/file.txt
7f2a36614000-7f2a36615000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36615000-7f2a36635000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36635000-7f2a3663d000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663d000-7f2a3663e000 rw-p 00000000 00:00 0
7f2a3663e000-7f2a3663f000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663f000-7f2a36640000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36640000-7f2a36641000 rw-p 00000000 00:00 0
7ffe92d0c000-7ffe92d2d000 rw-p 00000000 00:00 0 [stack]
7ffe92db3000-7ffe92db7000 r--p 00000000 00:00 0 [vvar]
7ffe92db7000-7ffe92db9000 r-xp 00000000 00:00 0 [vdso]
```

[CHILD] Virtual Memory Map:

Virtual Memory Map of process [1625529]:

```
55be8a838000-55be8a839000 r--p 00000000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a839000-55be8a83a000 r-xp 00001000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83a000-55be8a83b000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83b000-55be8a83c000 r--p 00003000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83c000-55be8a83d000 rw-p 00004000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8b80d000-55be8b82e000 rw-p 00000000 00:00 0 [heap]
7f2a36438000-7f2a3645a000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a3645a000-7f2a365b3000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a365b3000-7f2a36602000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36602000-7f2a36606000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36606000-7f2a36608000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36608000-7f2a3660e000 rw-p 00000000 00:00 0
7f2a36612000-7f2a36613000 rw-s 00000000 00:01 10429 /dev/zero (deleted)
7f2a36613000-7f2a36614000 r--p 00000000 00:27 1444948 /home/oslab/oslab067/lab3_exercise/PartA/mmap/file.txt
7f2a36614000-7f2a36615000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36615000-7f2a36635000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36635000-7f2a3663d000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663d000-7f2a3663e000 rw-p 00000000 00:00 0
7f2a3663e000-7f2a3663f000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663f000-7f2a36640000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36640000-7f2a36641000 rw-p 00000000 00:00 0
7ffe92d0c000-7ffe92d2d000 rw-p 00000000 00:00 0 [stack]
7ffe92db3000-7ffe92db7000 r--p 00000000 00:00 0 [vvar]
7ffe92db7000-7ffe92db9000 r-xp 00000000 00:00 0 [vdso]
```

angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cs.lab.ece.ntua.gr — 134x60

Step 8: Find the physical address of the private heap buffer (main) for both the parent and the child.

[PARENT] Physical address of heap_private_buf: 0x1d96df000

[CHILD] Physical address of heap_private_buf: 0x1d96df000

Step 9: Write to the private buffer from the child and repeat step 8. What happened?

[PARENT] Physical address of heap_private_buf: 0x1d96df000

[CHILD] Physical address of heap_private_buf AFTER write: 0x17b580000

Step 10: Write to the shared heap buffer (main) from child and get the physical address for both the parent and the child. What happened?

[PARENT] Physical address of heap_shared_buf: 0x1c1fae000

[CHILD] Physical address of heap_shared_buf AFTER write: 0x1c1fae000

```
angeloskamariadis — oslab067@os-node1: ~/lab3_exercise/PartA/mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 134x60
Step 11: Disable writing on the shared buffer for the child. Verify through the maps for the parent and the child.

[PARENT] Verifying maps (Parent should still have rw- permissions):
Virtual Memory Map of process [1625527]:
55be8a838000-55be8a839000 r--p 00000000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a839000-55be8a83a000 r-xp 00001000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83a000-55be8a83b000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83b000-55be8a83c000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83c000-55be8a83d000 rw-p 00003000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8b80d000-55be8b82e000 rw-p 00000000 00:00 0 [heap]
7f2a36438000-7f2a3645a000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a3645a000-7f2a365b3000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a365b3000-7f2a36602000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36602000-7f2a36606000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36606000-7f2a36608000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36608000-7f2a3660e000 rw-p 00000000 00:00 0
7f2a36612000-7f2a36613000 rw-s 00000000 00:01 10429 /dev/zero (deleted)
7f2a36613000-7f2a36614000 r--p 00000000 00:27 1444948 /home/oslab/oslab067/lab3_exercise/PartA/mmap/file.txt
7f2a36614000-7f2a36615000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36615000-7f2a36635000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36635000-7f2a3663d000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663d000-7f2a3663e000 rw-p 00000000 00:00 0
7f2a3663e000-7f2a3663f000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663f000-7f2a36640000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36640000-7f2a36641000 rw-p 00000000 00:00 0
7ffe92d0c000-7ffe92d2d000 rw-p 00000000 00:00 0 [stack]
7ffe92db3000-7ffe92db7000 r--p 00000000 00:00 0 [vvar]
7ffe92db7000-7ffe92db9000 r-xp 00000000 00:00 0 [vdso]
-----

[CHILD] Verifying maps (Child should now have r-- permissions on shared buffer):
Virtual Memory Map of process [1625529]:
55be8a838000-55be8a839000 r--p 00000000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a839000-55be8a83a000 r-xp 00001000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83a000-55be8a83b000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83b000-55be8a83c000 r--p 00002000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8a83c000-55be8a83d000 rw-p 00003000 00:27 1444247 /home/oslab/oslab067/lab3_exercise/PartA/mmap/mmap
55be8b80d000-55be8b82e000 rw-p 00000000 00:00 0 [heap]
7f2a36438000-7f2a3645a000 r--p 00000000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a3645a000-7f2a365b3000 r-xp 00022000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a365b3000-7f2a36602000 r--p 0017b000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36602000-7f2a36606000 r--p 001c9000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36606000-7f2a36608000 rw-p 001cd000 fe:01 144567 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f2a36608000-7f2a3660e000 rw-p 00000000 00:00 0
7f2a36612000-7f2a36613000 r--s 00000000 00:01 10429 /dev/zero (deleted)
7f2a36613000-7f2a36614000 r--p 00000000 00:27 1444948 /home/oslab/oslab067/lab3_exercise/PartA/mmap/file.txt
7f2a36614000-7f2a36615000 r--p 00000000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36615000-7f2a36635000 r-xp 00001000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36635000-7f2a3663d000 r--p 00021000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663d000-7f2a3663e000 rw-p 00000000 00:00 0
7f2a3663e000-7f2a3663f000 r--p 00029000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a3663f000-7f2a36640000 rw-p 0002a000 fe:01 144563 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f2a36640000-7f2a36641000 rw-p 00000000 00:00 0
7ffe92d0c000-7ffe92d2d000 rw-p 00000000 00:00 0 [stack]
7ffe92db3000-7ffe92db7000 r--p 00000000 00:00 0 [vvar]
7ffe92db7000-7ffe92db9000 r-xp 00000000 00:00 0 [vdso]
```

Απαντήσεις στις ερωτήσεις

■ Βήμα 3

Στο βήμα 3, παρατηρούμε ότι η φυσική διεύθυνση του buffer είναι 0, παρόλο που η κλήση συστήματος `mmap()` μας έχει δώσει μια έγκυρη εικονική διεύθυνση, γεγονός που οφείλεται στον μηχανισμό Demand Paging του λειτουργικού συστήματος. Συγκεκριμένα, το Linux εφαρμόζει lazy allocation και έτσι, κατά την εκτέλεση της `mmap()`, το σύστημα απλώς δεσμεύει τον εικονικό χώρο διευθύνσεων και ενημερώνει τα page tables, αλλά δεν εκχωρεί άμεσα πραγματική μνήμη RAM για να κάνει οικονομία πόρων. Εφόσον μέχρι αυτό το σημείο δεν έχουμε προσπελάσει τον buffer, δηλαδή δεν έχουμε γράψει ή διαβάσει δεδομένα, το λειτουργικό σύστημα δεν τον έχει αντιστοιχίσει στη φυσική μνήμη. Μόνο όταν το πρόγραμμα προσπαθήσει να γράψει σε αυτόν, το hardware θα εντοπίσει την απουσία της σελίδας προκαλώντας ένα Page Fault, το οποίο θα αναγκάσει το Λειτουργικό Σύστημα να βρει και να εκχωρήσει εκείνη ακριβώς τη στιγμή μια πραγματική φυσική σελίδα στη διεργασία.

▪ Βήμα 4

Στο Βήμα 4 παρατηρούμε ότι, μετά την αρχικοποίηση του buffer με μηδενικά, η συνάρτηση "get_physical_address()" δεν επιστρέφει πλέον 0, αλλά μια πραγματική, μη μηδενική δεκαεξαδική τιμή για τη φυσική διεύθυνση. Αυτή η αλλαγή συμβαίνει επειδή η εντολή memset ανάγκασε τη CPU να προσπαθήσει να γράψει δεδομένα σε μια εικονική σελίδα η οποία δεν είχε ακόμη αντιστοιχιστεί στη μνήμη RAM. Η απόπειρα αυτή προκάλεσε ένα Page Fault στο hardware. Το λειτουργικό σύστημα έπιασε αυτή τη διακοπή, εκτέλεσε τον μηχανισμό Demand Paging, βρήκε ένα ελεύθερο frame στην κύρια μνήμη, το εκχώρησε αποκλειστικά στη διεργασία μας και τέλος, ενημέρωσε τα Page Tables, ώστε πλέον η εικονική διεύθυνση του buffer να αντιστοιχεί στην πραγματική αυτή φυσική διεύθυνση.

▪ Βήμα 7

Στο Βήμα 7, παρατηρούμε ότι οι χάρτες εικονικής μνήμης (virtual memory maps) της γονικής διεργασίας και της διεργασίας παιδιού, είναι σχεδόν ίδιοι. Οι εικονικές διευθύνσεις στις οποίες έχουν δεσμευτεί οι buffers (private και shared), καθώς και η απεικόνιση του αρχείου file.txt, είναι ακριβώς οι ίδιες και για τις δύο διεργασίες. Αυτό εξηγείται από τον τρόπο λειτουργίας της κλήσης συστήματος fork(). Όταν εκτελείται η fork(), το λειτουργικό σύστημα δημιουργεί τη νέα διεργασία (το παιδί) φτιάχνοντας ένα ακριβές αντίγραφο του εικονικού χώρου διευθύνσεων του γονέα. Επομένως, το παιδί κληρονομεί την ίδια ακριβώς εικονική διάταξη μνήμης τη στιγμή της δημιουργίας του, για αυτό και βλέπουμε τις ίδιες εικονικές διευθύνσεις στις εκτυπώσεις, παρόλο που πλέον αποτελούν δύο εντελώς ανεξάρτητες διεργασίες. Δηλαδή, η fork() έκανε copy - paste τον χάρτη του γονέα και τον έδωσε στο παιδί.

▪ Βήμα 8

Στο Βήμα 8 παρατηρούμε, ότι αμέσως μετά την κλήση της fork(), η φυσική διεύθυνση του private buffer είναι ακριβώς η ίδια και για τη γονική διεργασία και για τη διεργασία παιδί. Αυτό το φαινόμενο αποτελεί την πρώτη φάση του μηχανισμού CoW, που εφαρμόζει το λειτουργικό σύστημα. Προκειμένου να εξοικονομήσει χρόνο και πολύτιμη μνήμη RAM, η fork() δεν δημιουργεί άμεσα ένα πραγματικό, φυσικό αντίγραφο όλης της μνήμης του γονέα για το νέο παιδί. Αντίθετα, το σύστημα ρυθμίζει τα Page Tables και των δύο διεργασιών ώστε οι ίδιες εικονικές διευθύνσεις να δείχνουν στο ίδιο ακριβώς frame. Παράλληλα, το ΛΣ μαρκάρει αυτήν τη σελίδα προσωρινά ως read only στο hardware. Εφόσον στο συγκεκριμένο βήμα καμία από τις δύο διεργασίες δεν έχει επιχειρήσει ακόμα να τροποποιήσει τα δεδομένα του private buffer, συνεχίζουν να μοιράζονται την ίδια φυσική μνήμη.

▪ Βήμα 9

Στο Βήμα 9 παρατηρούμε, ότι μόλις η διεργασία παιδί προσπαθήσει να γράψει δεδομένα στον private buffer, η φυσική της διεύθυνση αλλάζει αποκτώντας μια νέα τιμή, ενώ αντίθετα η φυσική διεύθυνση του γονέα παραμένει η ίδια με πριν. Εδώ βλέπουμε την ολοκλήρωση του μηχανισμού CoW στην πράξη. Η απόπειρα εγγραφής του παιδιού σε μια σελίδα που το λειτουργικό σύστημα είχε μαρκάρει προσωρινά ως read only (κατά την εκτέλεση της fork()), προκάλεσε ένα CoW Page Fault στο hardware. Το λειτουργικό σύστημα επενέβη αμέσως, διέθεσε ένα νέο, ανεξάρτητο frame στη μνήμη RAM αποκλειστικά για το παιδί, αντέγραψε τα δεδομένα της σελίδας σε αυτό, ενημέρωσε τα Page Tables του παιδιού και του έδωσε

δικαιώματα εγγραφής. Πλέον, ενώ και οι δύο διεργασίες εξακολουθούν να χρησιμοποιούν την ίδια εικονική διεύθυνση για τον private buffer, αυτές αντιστοιχούν σε εντελώς διαφορετικές φυσικές σελίδες, εξασφαλίζοντας έτσι την πλήρη απομόνωση των δεδομένων των δύο διεργασιών.

▪ Βήμα 10

Στο Βήμα 10 παρατηρούμε, ότι σε πλήρη αντίθεση με τον private buffer, όταν το παιδί γράφει δεδομένα στον shared buffer, η φυσική του διεύθυνση δεν αλλάζει και παραμένει ακριβώς η ίδια τόσο για τη γονική διεργασία όσο και για το παιδί. Αυτό συμβαίνει επειδή κατά τη δέσμευση αυτού του buffer χρησιμοποιήσαμε τη σημαία MAP_SHARED στην κλήση mmap(). Η σημαία αυτή ειδοποιεί το λειτουργικό σύστημα πως η συγκεκριμένη περιοχή προορίζεται για διαμοιρασμό (Shared Memory), με αποτέλεσμα να μην εφαρμόζεται ο μηχανισμός CoW. Αντίθετα, τα Page Tables και των δύο διεργασιών ρυθμίζονται ώστε να δείχνουν μόνιμα στο ίδιο, μοναδικό frame στη RAM, επιτρέποντας αμφίδρομη εγγραφή και ανάγνωση. Έτσι, οποιαδήποτε τροποποίηση κάνει η μία διεργασία γίνεται ακαριαία ορατή και στην άλλη.

Άσκηση 1.2 - Παράλληλος Υπολογισμός Mandelbrot με Διεργασίες Αντί για Νήματα

Στο παρόν μέρος της 3^{ης} εργαστηριακής άσκησης, τροποποιήθηκε το πρόγραμμα υπολογισμού του Mandelbrot set, έτσι ώστε η παραλληλοποίηση να γίνεται με διεργασίες, αντί για νήματα. Η κατανομή των γραμμών έγινε κυκλικά ως εξής: η διεργασία i υπολογίζει τις γραμμές i , $i + NPROCS$, $i + 2*NPROCS$, κ.ο.κ. Υλοποιήθηκαν δύο εκδόσεις του προγράμματος, μία με POSIX Semaphores σε διαμοιραζόμενη μνήμη και μία χωρίς Semaphores, με χρήση shared output buffer.

1.2.1 - Semaphores Πάνω από Διαμοιραζόμενη Μνήμη

Στην πρώτη υλοποίηση (υλοποίηση) δημιουργούνται NPROCS διεργασίες με fork(). Κάθε διεργασία αναλαμβάνει συγκεκριμένες γραμμές του Mandelbrot set. Για παράδειγμα, η διεργασία i υπολογίζει τις γραμμές i , $i + NPROCS$, $i + 2*NPROCS$, κ.ο.κ. Επειδή πολλές διεργασίες μπορούν να ολοκληρώσουν τον υπολογισμό των γραμμών τους με διαφορετική σειρά, απαιτείται συγχρονισμός ώστε η εκτύπωση στο terminal να γίνει με τη σωστή σειρά γραμμών. Για αυτό χρησιμοποιήθηκαν POSIX semaphores. Οι semaphores τοποθετήθηκαν σε διαμοιραζόμενη μνήμη, η οποία δημιουργείται με mmap() και τα flags MAP_SHARED | MAP_ANONYMOUS. Έτσι οι semaphores είναι ορατοί από όλες τις διεργασίες παιδιά μετά το fork(). Το δεύτερο όρισμα της sem_init() είναι 1, επειδή οι semaphores χρησιμοποιούνται για συγχρονισμό μεταξύ διαφορετικών διεργασιών και όχι μεταξύ threads της ίδιας διεργασίας. Όταν το pshared είναι 1, ο semaphore μπορεί να χρησιμοποιηθεί από πολλές διεργασίες, αρκεί να βρίσκεται σε διαμοιραζόμενη μνήμη (shared memory). Αν η τιμή ήταν 0, τότε ο semaphore θα μπορούσε να χρησιμοποιηθεί μόνο μεταξύ threads της ίδιας διεργασίας.

Ο τροποποιημένος από εμάς κώδικας, παρουσιάζεται παρακάτω.

```
angeloskamariadis — oslab067@os-node2: ~/lab3_exercise/PartB/sync-mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 139x60
#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>

#include "mandel-lib.h"

#define MANDEL_MAX_ITERATION 100000

int y_chars = 50;
int x_chars = 90;

double xmin = -1.8, xmax = 1.8;
double ymin = -1.0, ymax = 1.0;

double xstep;
double ystep;

void compute_mandel_line(int line, int color_val[])
{
    double x, y;
    int n;
    int val;

    y = ymax - ystep * line;

    for (x = xmin, n = 0; n < x_chars; x += xstep, n++) {
        val = mandel_iterations_at_point(x, y, MANDEL_MAX_ITERATION);
        if (val > 255)
            val = 255;

        val = xterm_color(val);
        color_val[n] = val;
    }
}

void output_mandel_line(int fd, int color_val[])
{
    int i;
    char point = '@';
    char newline = '\n';

    for (i = 0; i < x_chars; i++) {
        set_xterm_color(fd, color_val[i]);
        if (write(fd, &point, 1) != 1) {
            perror("output_mandel_line: write point");
            exit(1);
        }
    }

    if (write(fd, &newline, 1) != 1) {
        perror("output_mandel_line: write newline");
        exit(1);
    }
}

}

void compute_and_output_mandel_line(int fd, int line)
{
    int color_val[x_chars];

    compute_mandel_line(line, color_val);
    output_mandel_line(fd, color_val);
}

void *create_shared_memory_area(unsigned int numbytes)
{
    int pages;
    void *addr;
    long page_size;

    if (numbytes == 0) {
        fprintf(stderr, "%s: internal error: called for numbytes == 0\n", __func__);
        exit(1);
    }

    page_size = sysconf(_SC_PAGE_SIZE);
    pages = (numbytes - 1) / page_size + 1;

    addr = mmap(NULL,
                pages * page_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED | MAP_ANONYMOUS,
                -1,
                0);

    if (addr == MAP_FAILED) {
        perror("create_shared_memory_area: mmap failed");
        exit(1);
    }

    return addr;
}

void destroy_shared_memory_area(void *addr, unsigned int numbytes)
{
    int pages;
    long page_size;

    if (numbytes == 0) {
        fprintf(stderr, "%s: internal error: called for numbytes == 0\n", __func__);
        exit(1);
    }

    page_size = sysconf(_SC_PAGE_SIZE);
    pages = (numbytes - 1) / page_size + 1;

    if (munmap(addr, pages * page_size) == -1) {
        perror("destroy_shared_memory_area: munmap failed");
        exit(1);
    }
}

int main(int argc, char **argv)
```



```
angeloskamariadis — oslab067@os-node2: ~/lab3_exercise/PartB/sync-mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 139x60
{
    int line;
    int i;
    int nprocs;
    pid_t p;
    sem_t *line_sems;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s NPROCS\n", argv[0]);
        exit(1);
    }

    nprocs = atoi(argv[1]);
    if (nprocs <= 0) {
        fprintf(stderr, "NPROCS must be a positive integer\n");
        exit(1);
    }

    xstep = (xmax - xmin) / x_chars;
    ystep = (ymax - ymin) / y_chars;

    line_sems = create_shared_memory_area(y_chars * sizeof(sem_t));

    for (line = 0; line < y_chars; line++) {
        if (sem_init(&line_sems[line], 1, line == 0 ? 1 : 0) == -1) {
            perror("sem_init");
            exit(1);
        }
    }

    for (i = 0; i < nprocs; i++) {
        p = fork();

        if (p < 0) {
            perror("fork");
            exit(1);
        }

        if (p == 0) {
            int my_id = i;

            for (line = my_id; line < y_chars; line += nprocs) {
                int color_val[x_chars];

                compute_mandel_line(line, color_val);

                if (sem_wait(&line_sems[line]) == -1) {
                    perror("sem_wait");
                    exit(1);
                }

                output_mandel_line(1, color_val);

                if (line + 1 < y_chars) {
                    if (sem_post(&line_sems[line + 1]) == -1) {
                        perror("sem_post");
                        exit(1);
                    }
                }
            }

            exit(0);
        }
    }

    for (i = 0; i < nprocs; i++) {
        if (wait(NULL) == -1) {
            perror("wait");
            exit(1);
        }
    }

    reset_xterm_color(1);

    for (line = 0; line < y_chars; line++) {
        if (sem_destroy(&line_sems[line]) == -1) {
            perror("sem_destroy");
            exit(1);
        }
    }

    destroy_shared_memory_area(line_sems, y_chars * sizeof(sem_t));

    return 0;
}
```

Εκτελώντας τον κώδικα, πήραμε το ακόλουθο output.

■ Ερώτηση 2

Το `mmap()` μπορεί να χρησιμοποιηθεί για διαμοιρασμό μνήμης μεταξύ διεργασιών που δεν έχουν κοινό ancestor, αλλά όχι με απλό anonymous mapping. Το `MAP_SHARED` | `MAP_ANONYMOUS` λειτουργεί πρακτικά όταν οι διεργασίες σχετίζονται μέσω `fork()`, επειδή το παιδί κληρονομεί το mapping. Για άσχετες διεργασίες χρειάζεται κοινό file-backed mapping ή POSIX shared memory object μέσω `shm_open()`, ώστε όλες οι διεργασίες να μπορούν να ανοίξουν το ίδιο αντικείμενο και να το κάνουν `mmap()` με `MAP_SHARED`.

1.2.2 - Υλοποίηση χωρίς *semaphores*

Στη δεύτερη υλοποίηση αποφεύγεται η χρήση *semaphores*. Οι διεργασίες παιδιά δεν τυπώνουν απευθείας στο terminal. Αντίθετα, κάθε παιδί υπολογίζει τις γραμμές που του αντιστοιχούν και τις αποθηκεύει σε έναν κοινό δισδιάστατο buffer στη διαμοιραζόμενη μνήμη. Ο buffer έχει μέγεθος `y_chars` x `x_chars`, δηλαδή περιέχει όλες τις γραμμές εξόδου του Mandelbrot. Κάθε διεργασία γράφει σε διαφορετικές γραμμές, άρα δεν υπάρχει σύγκρουση εγγραφών. Ο γονέας περιμένει να ολοκληρωθούν όλες οι διεργασίες με `wait()` και στη συνέχεια τυπώνει ο ίδιος όλες τις γραμμές με τη σωστή σειρά.

Στην υλοποίηση αυτή ο συγχρονισμός επιτυγχάνεται μέσω της `wait()`. Ο parent τυπώνει το αποτέλεσμα μόνο αφού ολοκληρωθούν όλα τα παιδιά. Επομένως η `wait()` λειτουργεί σαν barrier.

Ο τροποποιημένος από εμάς κώδικας, παρουσιάζεται παρακάτω.

```
angeloskamariadis — oslab067@os-node2: ~/lab3_exercise/PartB/sync-mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 139x60
#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/wait.h>

#include "mandel-lib.h"

#define MANDEL_MAX_ITERATION 100000

int y_chars = 50;
int x_chars = 90;

double xmin = -1.8, xmax = 1.0;
double ymin = -1.0, ymax = 1.0;

double xstep;
double ystep;

void compute_mandel_line(int line, int color_val[])
{
    double x, y;
    int n;
    int val;

    y = ymax - ystep * line;

    for (x = xmin, n = 0; n < x_chars; x += xstep, n++) {
        val = mandel_iterations_at_point(x, y, MANDEL_MAX_ITERATION);
        if (val > 255)
            val = 255;

        val = xterm_color(val);
        color_val[n] = val;
    }
}

void output_mandel_line(int fd, int color_val[])
{
    int i;
    char point = '@';
    char newline = '\n';

    for (i = 0; i < x_chars; i++) {
        set_xterm_color(fd, color_val[i]);
        if (write(fd, &point, 1) != 1) {
            perror("output_mandel_line: write point");
            exit(1);
        }
    }

    if (write(fd, &newline, 1) != 1) {
        perror("output_mandel_line: write newline");
        exit(1);
    }
}
```



```

void *create_shared_memory_area(unsigned int numbytes)
{
    int pages;
    void *addr;
    long page_size;

    if (numbytes == 0) {
        fprintf(stderr, "%s: internal error: called for numbytes == 0\n", __func__);
        exit(1);
    }

    page_size = sysconf(_SC_PAGE_SIZE);
    pages = (numbytes - 1) / page_size + 1;

    addr = mmap(NULL,
                pages * page_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED | MAP_ANONYMOUS,
                -1,
                0);

    if (addr == MAP_FAILED) {
        perror("create_shared_memory_area: mmap failed");
        exit(1);
    }

    return addr;
}

void destroy_shared_memory_area(void *addr, unsigned int numbytes)
{
    int pages;
    long page_size;

    if (numbytes == 0) {
        fprintf(stderr, "%s: internal error: called for numbytes == 0\n", __func__);
        exit(1);
    }

    page_size = sysconf(_SC_PAGE_SIZE);
    pages = (numbytes - 1) / page_size + 1;

    if (munmap(addr, pages * page_size) == -1) {
        perror("destroy_shared_memory_area: munmap failed");
        exit(1);
    }
}

int main(int argc, char **argv)
{
    int line;
    int i;
    int nprocs;
    pid_t p;
    int *shared_output;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s NPROCS\n", argv[0]);
        exit(1);
    }

    nprocs = atoi(argv[1]);
    if (nprocs <= 0) {
        fprintf(stderr, "NPROCS must be a positive integer\n");
        exit(1);
    }

    xstep = (xmax - xmin) / x_chars;
    ystep = (ymax - ymin) / y_chars;

    shared_output = create_shared_memory_area(y_chars * x_chars * sizeof(int));

    for (i = 0; i < nprocs; i++) {
        p = fork();

        if (p < 0) {
            perror("fork");
            exit(1);
        }

        if (p == 0) {
            int my_id = i;

            for (line = my_id; line < y_chars; line += nprocs) {
                int *row;

                row = shared_output + line * x_chars;
                compute_mandel_line(line, row);
            }

            exit(0);
        }
    }

    for (i = 0; i < nprocs; i++) {
        if (wait(NULL) == -1) {
            perror("wait");
            exit(1);
        }
    }

    for (line = 0; line < y_chars; line++) {
        int *row;

        row = shared_output + line * x_chars;
        output_mandel_line(1, row);
    }

    reset_xterm_color(1);

    destroy_shared_memory_area(shared_output, y_chars * x_chars * sizeof(int));

    return 0;
}

```

Εκτελώντας τον κώδικα, πήραμε το ακόλουθο output.

```
oslab067@os-node2: ~/lab3_exercise/PartB/sync-mmap — ssh oslab067@orion.cslab.ece.ntua.gr — 139x60
oslab067@os-node2:~/lab3_exercise/PartB/sync-mmap$ ./mandel-fork-nosem 4 | tee mandel_fork_nosem_output.txt
```

Απαντήσεις στις ερωτήσεις

- Ερώτηση 1

Στην υλοποίηση χωρίς semaphores, ο συγχρονισμός γίνεται στο σημείο όπου ο parent καλεί wait() για όλες τις διεργασίες παιδιά. Τα παιδιά πρώτα γράφουν τα αποτελέσματά τους στον κοινό buffer και ο parent τυπώνει μόνο αφού ολοκληρωθούν όλα. Άρα η wait() λειτουργεί ως σημείο συγχρονισμού/barrier.

- Ερώτηση 2

Αν ο buffer είχε διαστάσεις NPROCS x x_chars, τότε κάθε διεργασία θα είχε μόνο μία γραμμή διαθέσιμη στον buffer. Όμως κάθε διεργασία υπολογίζει περισσότερες από μία γραμμές, δηλαδή i, i + NPROCS, i + 2*NPROCS κ.ο.κ. Επομένως θα έπρεπε να ξαναγράφει πάνω στην ίδια θέση του buffer, γάνοντας προηγούμενα αποτελέσματα.

Σε αυτή την περίπτωση ο parent δεν θα μπορούσε απλά να περιμένει μέχρι το τέλος και μετά να τυπώσει όλες τις γραμμές. Θα χρειαζόταν επιπλέον συγχρονισμός, ώστε κάθε γραμμή να τυπώνεται πριν αντικατασταθεί από επόμενη.

Έλεγχος κατά πόσο τα outputs είναι πλήρη:

```
oslab067@os-node2: ~/lab3_exercise/PartB/sync-mmap — ssh oslab067@orion.csilab.ece.ntua.gr — 139x60
oslab067@os-node2:~/lab3_exercise/PartB/sync-mmap$ wc -l mandel_fork_semaphores_output.txt
wc -l mandel_fork_nosem_output.txt
50 mandel_fork_semaphores_output.txt
50 mandel_fork_nosem_output.txt
```

Άσκηση 1.3 - Επέκταση της Άσκησης 1

Στο παρόν μέρος της 3^{ης} εργαστηριακής άσκησης, υλοποιήθηκε μία επέκταση του προγράμματος καταμέτρησης εμφανίσεων χαρακτήρα σε αρχείο. Αντί κάθε διεργασία παιδί να διατηρεί το αποτέλεσμα σε τοπική μεταβλητή και να το στέλνει στον parent μέσω pipe, όλες οι διεργασίες παιδιά ενημερώνουν έναν κοινό μετρητή που βρίσκεται σε διαμοιραζόμενη μνήμη. Το αρχείο εισόδου χωρίζεται σε nprocs τμήματα. Κάθε παιδί αναλαμβάνει ένα τμήμα του αρχείου και μετρά πόσες φορές εμφανίζεται ο ζητούμενος χαρακτήρας. Όταν βρει μία εμφάνιση, αυξάνει τον κοινό counter. Επειδή πολλές διεργασίες μπορεί να προσπαθήσουν να αυξήσουν τον counter ταυτόχρονα, η πρόσβαση προστατεύεται με POSIX semaphore. Η διαμοιραζόμενη μνήμη δημιουργείται με mmap() χρησιμοποιώντας τα flags MAP_SHARED | MAP_ANONYMOUS. Στην ίδια shared memory δομή αποθηκεύονται και ο κοινός counter και ο semaphore.

Ο τροποποιημένος από εμάς κώδικας, παρουσιάζεται παρακάτω.

```
angeloskamariadis — oslab067@os-node2: ~/lab3_exercise/PartC/char-count — ssh oslab067@orion.csilab.ece.ntua.gr — 139x60
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <signal.h>
#include <semaphore.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <sys/wait.h>

#define BUF_SIZE 4096

struct shared_data {
    sem_t sem;
    int counter;
};

static struct shared_data *shared = NULL;

void sigint_handler(int signo)
{
    (void) signo;

    if (shared != NULL) {
        printf("\nSIGINT: current shared counter = %d\n", shared->counter);
        fflush(stdout);
    }
}

void count_part(const char *filename, char target, off_t start, off_t end)
{
    int fd;
    char buf[BUF_SIZE];
    off_t pos;
    ssize_t nread;
    size_t to_read;
    int i;

    fd = open(filename, O_RDONLY);
    if (fd < 0) {
        perror("open child");
        exit(1);
    }

    pos = start;

    while (pos < end) {
        if (end - pos < BUF_SIZE)
            to_read = end - pos;
        else
            to_read = BUF_SIZE;

        nread = pread(fd, buf, to_read, pos);

        if (nread < 0) {
            if (errno == EINTR)
                continue;
        }
    }
}
```



```

        perror("pread");
        close(fd);
        exit(1);
    }

    if (nread == 0)
        break;

    for (i = 0; i < nread; i++) {
        if (buf[i] == target) {
            sem_wait(&shared->sem);
            shared->counter++;
            sem_post(&shared->sem);
        }
    }

    pos += nread;
}

close(fd);
}

int main(int argc, char *argv[])
{
    const char *input_file;
    const char *output_file;
    char target;
    int nprocs;
    int fd;
    FILE *fpw;
    struct stat st;
    off_t file_size;
    int i;
    pid_t pid;
    int status;

    if (argc != 5) {
        fprintf(stderr, "Usage: %s input_file output_file character nprocs\n", argv[0]);
        exit(1);
    }

    input_file = argv[1];
    output_file = argv[2];
    target = argv[3][0];
    nprocs = atoi(argv[4]);

    if (nprocs <= 0) {
        fprintf(stderr, "nprocs must be positive\n");
        exit(1);
    }

    fd = open(input_file, O_RDONLY);
    if (fd < 0) {
        perror("open input");
        exit(1);
    }

    if (fstat(fd, &st) < 0) {
        perror("fstat");

```

```

        close(fd);
        exit(1);
    }

    file_size = st.st_size;
    close(fd);

    shared = mmap(NULL,
                  sizeof(struct shared_data),
                  PROT_READ | PROT_WRITE,
                  MAP_SHARED | MAP_ANONYMOUS,
                  -1,
                  0);

    if (shared == MAP_FAILED) {
        perror("mmap");
        exit(1);
    }

    shared->counter = 0;

    if (sem_init(&shared->sem, 1, 1) < 0) {
        perror("sem_init");
        munmap(shared, sizeof(struct shared_data));
        exit(1);
    }

    signal(SIGINT, sigint_handler);

    for (i = 0; i < nprocs; i++) {
        pid = fork();

        if (pid < 0) {
            perror("fork");
            exit(1);
        }

        if (pid == 0) {
            off_t start;
            off_t end;

            signal(SIGINT, SIG_IGN);

            start = (file_size * i) / nprocs;
            end = (file_size * (i + 1)) / nprocs;

            count_part(input_file, target, start, end);

            exit(0);
        }
    }

    for (i = 0; i < nprocs; i++) {
        while (wait(&status) < 0) {
            if (errno == EINTR)
                continue;

            perror("wait");
            exit(1);
        }
    }
}

```

```

    }
}

fpw = fopen(output_file, "w");
if (fpw == NULL) {
    perror("fopen output");
    sem_destroy(&shared->sem);
    munmap(shared, sizeof(struct shared_data));
    exit(1);
}

fprintf(fpw,
        "The character '%c' appears %d times in file %s.\n",
        target,
        shared->counter,
        input_file);

fclose(fpw);

printf("The character '%c' appears %d times in file %s.\n",
        target,
        shared->counter,
        input_file);

sem_destroy(&shared->sem);
munmap(shared, sizeof(struct shared_data));

return 0;
}

```

Εκτελώντας τον κώδικα, πήραμε το ακόλουθο output.

```

oslab067@os-node2: ~/lab3_exercise/PartC/char-count — ssh oslab067@orion.cs.lab.ece.ntua.gr — 139x60
oslab067@os-node2:~/lab3_exercise/PartC/char-count$ ls
Makefile      a1.1-C.c      a1.3-shm      huge_input.txt  mandel-fork.c      output_shm.txt
Makefile.backup  a1.1-C.c.backup  a1.3-shm.c  input.txt      output_huge_shm.txt
oslab067@os-node2:~/lab3_exercise/PartC/char-count$ gcc -Wall -O2 -pthread -o a1.3-shm a1.3-shm.c
oslab067@os-node2:~/lab3_exercise/PartC/char-count$ ./a1.3-shm input.txt output_shm.txt a 4 | tee run_shm_output.txt
cat output_shm.txt
The character 'a' appears 4 times in file input.txt.
The character 'a' appears 4 times in file input.txt.

```

Εκτελώντας ξανά τον κώδικα για ένα test με Ctrl+C / SIGINT, πήραμε το παρακάτω output

```

oslab067@orion: ~ — ssh oslab067@orion.cs.lab.ece.ntua.gr — 139x60
oslab067@os-node2:~/lab3_exercise/PartC/char-count$ ./a1.3-shm huge_input.txt output_huge_shm.txt a 4
^C
SIGINT: current shared counter = 10785438

```

Συμπερασματικά, η επέκταση μας, δείχνει ότι οι διεργασίες παιδιά μπορούν να χρησιμοποιούν κοινή μεταβλητή σε διαμοιραζόμενη μνήμη για την καταμέτρηση των εμφανίσεων ενός χαρακτήρα. Η πρόσβαση στον κοινό counter προστατεύεται με semaphore, ώστε να αποφεύγονται race conditions. Επιπλέον, με τον χειρισμό του σήματος SIGINT, το πρόγραμμα μπορεί να εμφανίζει την τρέχουσα τιμή του counter κατά τη διάρκεια της εκτέλεσης, όπως ζητείται από την εκφώνηση.